BIFlow

Les 6 agents, scénario par scénario : ce que fait chacun, ce qu'il faut montrer et ce qu'il faut dire

Guide de la démonstration pour la validation technique et métier. Une partie par agent, pour que chaque membre du groupe présente le sien avec l'application, sur les vraies données NYC 311 (265 609 demandes, janvier 2023 à septembre 2026).

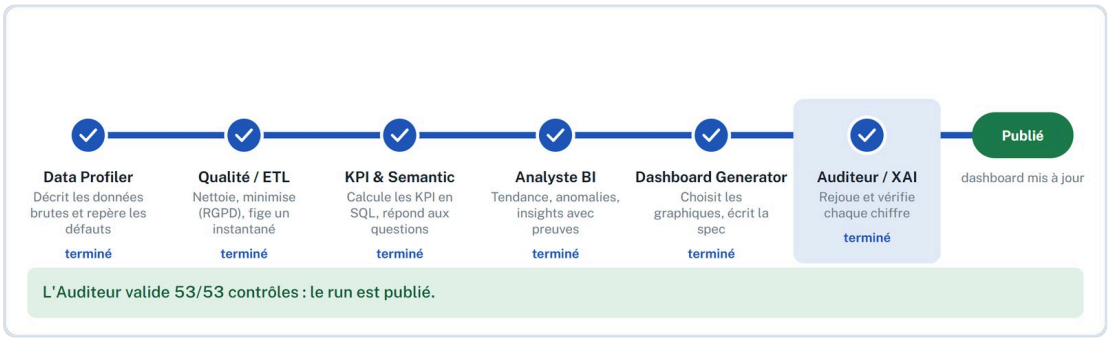


Agent	Ce qu'il fait	Présenté par
1 Data Profiler	décrit les données et repère les défauts	
2 Qualité / ETL	nettoie, protège les données, fige un instantané	
3 KPI & Semantic	calcule les KPI en SQL, répond aux questions	
4 Analyste BI	tendance, anomalies, insights avec preuves	
5 Dashboard Generator	choisit les graphiques, écrit la spécification	
6 Auditeur / XAI	rejoue et vérifie chaque chiffre avant publication	

Validation technique et métier, novembre 2026. Chiffres tirés du run publié `20261110T144646Z-b781b7` (LLM local qwen2.5:7b).

Le pipeline en une page

BIFlow part des données brutes d'un service public et produit un tableau de bord dont chaque chiffre est prouvé. Six agents travaillent toujours dans le même ordre, orchestrés par LangGraph. Le principe à répéter : **le LLM comprend et rédige ; le code calcule, vérifie et décide de publier.** Aucun chiffre publié ne vient du LLM.



Page Scénarios, run nominal terminé : chaque station est un agent ; « Publié » seulement si l'Auditeur valide.

Ce que l'orchestrateur garantit à chaque agent

Règle	Ce qui se passe
Validation	après chaque agent : sa sortie est contrôlée (schéma + contrôles métier). Les contrôles apparaissent en bas de chaque panneau d'agent.
Nouvel essai	une erreur ou une validation en échec : l'étape est relancée, jusqu'à 3 essais au total.
Échec	après le 3e essai : run **en échec**, les agents suivants ne tournent pas, rien n'est publié.
Renvoi de l'Auditeur	un écart trouvé par l'Auditeur renvoie le run vers l'étape fautive, **une seule fois** ; si l'écart persiste, le run est **non fiable**.
Trace	chaque essai de chaque agent est enregistré (statut, horaires, lignes en entrée et en sortie, contrôles) dans `run.json` et `audit.trace_log`.

Six scénarios, un par membre

Scénario	Agent	Ce qu'on fait	Résultat
1. Run nominal	1 Data Profiler	lancer le run du jour ; les 6 agents s'enchaînent	Publié 53/53
2. Panne d'un agent	2 Qualité / ETL	l'agent échoue à chaque essai : 3 essais, puis arrêt	Échec rien publié
3. Question métier et refus	3 KPI & Semantic	une question traduite en SQL par le LLM, puis une question hors périmètre refusée	15 443, puis refus motivé
4. Détection d'anomalie	4 Analyste BI	le pic du 24 février 2026 sur la courbe, avec ses preuves	460 au lieu de 218
5. Écart persistant	5 Dashboard Generator	l'Auditeur signale le tableau de bord, l'écart reste après le renvoi	Non fiable rien publié
6. Écart corrigé	6 Auditeur / XAI	l'Auditeur signale les KPI, renvoi, second audit réussi	Publié 2e audit

Les scénarios 1, 2, 5 et 6 se lancent depuis la page **Scénarios** ; le 3 se joue sur la page **Poser une question**, le 4 sur le **Tableau de bord**. Les pannes et les écarts sont **simulés** avec les mêmes options que la ligne de commande ; l'orchestrateur, les agents et les données sont les vrais.

Comment se répartir la présentation

Chaque membre présente **son agent** et **son scénario**, dans l'ordre du pipeline. Le run nominal lancé au début sert à tous : chacun clique sur l'onglet de son agent sous le schéma de ligne.

Déroulé conseillé (environ 20 minutes)

Temps	Qui	Action
0:00	Agent 1	Page Scénarios, *Run nominal*, « Ralentir » coché, **Lancer le scénario**. Pendant le run (environ 1 min 30 avec le LLM), présenter le pipeline et la règle « le LLM rédige, le code vérifie ».
1:30	Agent 1	Onglet **1 Data Profiler** : les défauts trouvés.
3:30	Agent 2	Onglet **2 Qualité / ETL**, puis lancer le scénario *Panne d'un agent* (20 s).
6:00	Agent 3	Onglet **3 KPI & Semantic**, puis scénario 3 sur la page *Poser une question*.
8:30	Agent 4	Onglet **4 Analyste BI**, puis scénario 4 sur la courbe du *Tableau de bord*.
11:00	Agent 5	Onglet **5 Dashboard Generator**, puis lancer le scénario 5 *Écart persistant* sur Dashboard Generator (environ 1 min 40) ; montrer le *Tableau de bord* inchangé.
14:30	Agent 6	Lancer le scénario 6 *Écart corrigé* sur KPI & Semantic (environ 2 min 30) ; pendant le run, commenter l'onglet **6 Auditeur / XAI** du run nominal.

Avant la séance

- [ ] `.\scripts\tasks.ps1 web --llm ollama`, puis `http://localhost:8000` ; en haut à droite : « LLM ollama:qwen2.5:7b ».
- [ ] Préchauffer le LLM : page *Poser une question*, cliquer sur une question d'exemple (la première réponse prend 15 à 20 s).
- [ ] Chacun a lu sa partie et sait cliquer sur son onglet ; chacun connaît **une** phrase sur les agents voisins (ce qu'il reçoit, à qui il passe).
- [ ] Présentation ouverte en mode diaporama sur la même machine (Alt+Tab vers l'application).
- [ ] Plan B : la vidéo `livrables/BIFlow_demo.mp4` (4 min, intégrée à la présentation) ; sans LLM : `.\scripts\tasks.ps1 web` (runs d'environ 10 s).

Trois phrases que tout le monde doit savoir dire

> « Le LLM comprend la question et rédige les textes ; tous les chiffres viennent d'une requête SQL enregistrée. »
>
> « Rien n'est publié sans l'Auditeur : il rejoue chaque requête, et si un écart persiste, le tableau de bord garde le dernier run validé. »
>
> « Les données personnelles sont retirées ou masquées : texte libre supprimé, coordonnées arrondies, aucun résultat sur moins de 10 demandes. »

La présentation, slide par slide

Le mail du professeur demande une présentation « simple » qui résume **le projet, ses objectifs, l'approche proposée et les principaux résultats**, puis l'application en démonstration. La présentation (`livrables/BIFlow_Validation.pptx`, 28 slides) suit exactement cet ordre. Chaque slide a son texte dans les notes du présentateur.

Qui présente quoi (proposition, à adapter)

Slides	Partie du mail	Contenu	Qui	Durée
1 à 7	Le projet	titre, plan, le problème (22,9 M de demandes NYC 311), démarche, utilisateurs, données réelles, « Comment pourrions-nous… »	membre 1 (Data Profiler)	3 min
8 à 10	Objectifs	les cinq preuves mesurables, les choix techniques comparés, « le LLM rédige, le code vérifie »	membre 2 (Qualité / ETL)	2 min
11 à 14	Approche proposée	les 6 agents et LangGraph, le rôle de chacun, sécurité et RGPD, l'application web	membre 3 (KPI)	3 min
15 à 18	Démonstration	vidéo de secours, tableau des 6 scénarios, puis les 6 scénarios en direct (voir « Le jour J »)	les 6 membres, un scénario chacun	16 à 20 min
19 à 22	Résultats techniques	les cinq preuves atteintes, l'apport du LLM (88 %), le test réel (53/53), ce que les tests ont corrigé	membre 4 (Analyste)	3 min
23 à 25	Résultats commerciaux	la valeur pour un service public, le marché, la concurrence	membre 5 (Dashboard)	2 min 30
26 à 28	Résultats commerciaux	le modèle économique, la feuille de route, la conclusion	membre 6 (Auditeur)	2 min 30

Ordre de la séance

1. **Slides 1 à 14** (environ 8 min) : projet, objectifs, approche.
2. **Slide 16** à l'écran pour annoncer les 6 scénarios, puis **basculer sur l'application** (Alt+Tab) : chaque membre joue son scénario.
3. **Slides 17 et 18** pour récapituler les scénarios en 30 s, si le temps le permet.
4. **Slides 19 à 28** (environ 8 min) : résultats techniques, puis partie commerciale, puis conclusion.
5. Questions du jury : chacun répond sur son agent (voir les questions préparées dans sa partie).

> **Si le temps est court :** passer les slides 4, 9 et 22, et ne jouer en direct que les scénarios 1, 2 et 3 ; montrer les scénarios 5 et 6 dans la vidéo (slide 15, à partir de 2 min 08).

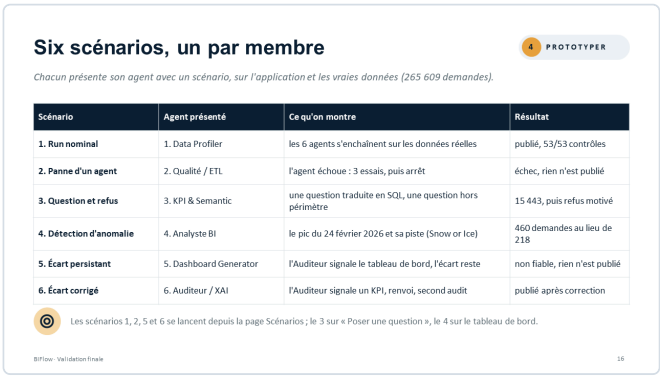
Ce qui a changé dans la présentation

- [ ] Plan (slide 2) avec les mots du mail : projet, objectifs, approche, démonstration, résultats.
- [ ] Nouvelles slides 16 à 18 : les 6 scénarios, un par membre, chacun expliqué en 3 lignes.
- [ ] Nouvelles slides 24 à 27 : marché, concurrence, modèle économique, feuille de route (validation commerciale).
- [ ] Slide « Limites assumées et pistes d'amélioration » retirée. Si le jury demande les limites, réponse courte page « Questions transverses ».

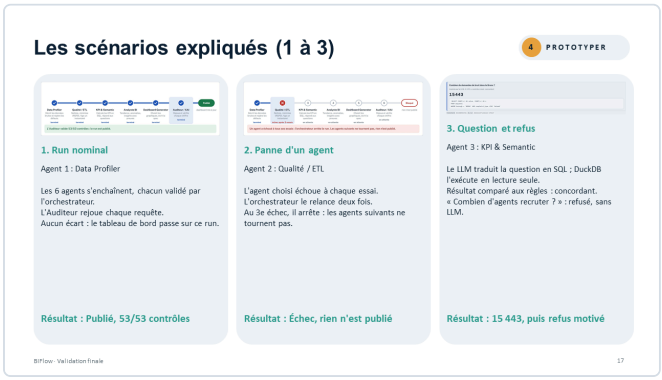
Les nouvelles slides en un coup d'œil



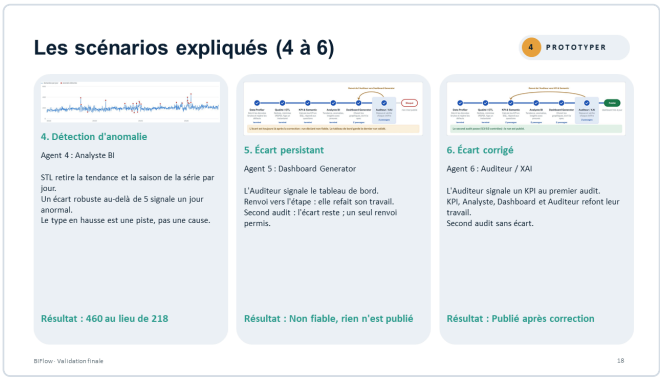
Slide 2 : le plan, dans les termes du mail.



Slide 16 : les 6 scénarios, un par membre.



Slide 17 : scénarios 1 à 3 expliqués.



Slide 18 : scénarios 4 à 6 expliqués.



Slide 24 : le marché, qui décide de l'achat.



Slide 25 : la concurrence.



Slide 26 : le modèle économique (prix = hypothèses).



Slide 27 : la feuille de route.

À savoir pour la partie commerciale : les prix de la slide 26 (pilote de 15 à 25 k€, abonnement de 2 à 5 k€ par mois) sont des **hypothèses**, pas des résultats. Si le jury demande d'où ils viennent : « ce sont nos hypothèses de départ, que le pilote doit confronter au budget réel d'une collectivité ».

Le jour J : comment on montre tout au professeur

Une seule machine, branchée au projecteur, avec l'application ouverte en plein écran. Chaque membre prend la souris pour **son** scénario : il montre et il explique. Les autres ne parlent pas pendant ce temps, sauf si le professeur leur pose une question.

Avant la séance

Quand	Quoi	Vérification
La veille	Chacun lit sa partie de ce guide et répète son texte une fois à voix haute, avec le chronomètre.	moins de 3 min
La veille	`.\.venv\Scripts\python -m pytest -q` sur la machine de démonstration.	aucun échec
30 min avant	Lancer Ollama, puis `.\scripts\tasks.ps1 web --llm ollama`.	« Uvicorn running »
25 min avant	Ouvrir `http://localhost:8000` en plein écran (F11), zoom du navigateur à 110 %.	bandeau « LLM ollama:qwen2.5:7b »
20 min avant	Préchauffer le LLM : page *Poser une question*, cliquer une question d'exemple.	réponse affichée
15 min avant	Lancer une fois le *Run nominal* pour que le tableau de bord montre un run du jour.	« Publié »
5 min avant	Revenir sur la page **Scénarios**, sans run affiché ; ouvrir la vidéo dans un autre onglet (plan B).	schéma de ligne vide

Pendant la démonstration : 4 règles

1. **Un seul run à la fois.** Le bouton affiche « Run en cours… » : attendre la fin avant d'en lancer un autre.
2. **Les scénarios longs se lancent au début de la partie** (5 et 6) : on parle pendant que les agents travaillent.
3. **On montre l'écran, puis on explique :** pointer la station qui s'allume, le journal, le résultat final.
4. **On termine chaque partie par une phrase-résultat** : « publié », « rien n'est publié », « refusé ».

Plans B

Si…	Alors
Le LLM est trop lent ou Ollama ne démarre pas	Fermer le serveur (Ctrl+C) et relancer `.\scripts\tasks.ps1 web` : les runs prennent environ 10 s, les questions du catalogue sont traitées par les règles. Le dire : « sans LLM, tout fonctionne avec la baseline ».
« Un run est déjà en cours »	Attendre la fin du run affiché ; la page *Historique* montre les runs terminés.
L'application ne répond plus	Passer la vidéo `livrables/BIFlow_demo.mp4` (4 min, elle montre les mêmes scénarios), puis répondre aux questions.
Le temps manque	Scénario 6 : montrer la vidéo à partir de 2 min 08 (scénario 5 à 2 min 37) au lieu de lancer le run.

Les 6 scénarios en détail

Pour chaque scénario : ce qui se passe dans le système, ce qu'on voit à l'écran, et comment le montrer au professeur, clic par clic.

1 Run nominal

Agent 1, Data Profiler. Page Scénarios. Environ 1 min 30 avec le LLM. Résultat : Publié

Comment ça marche

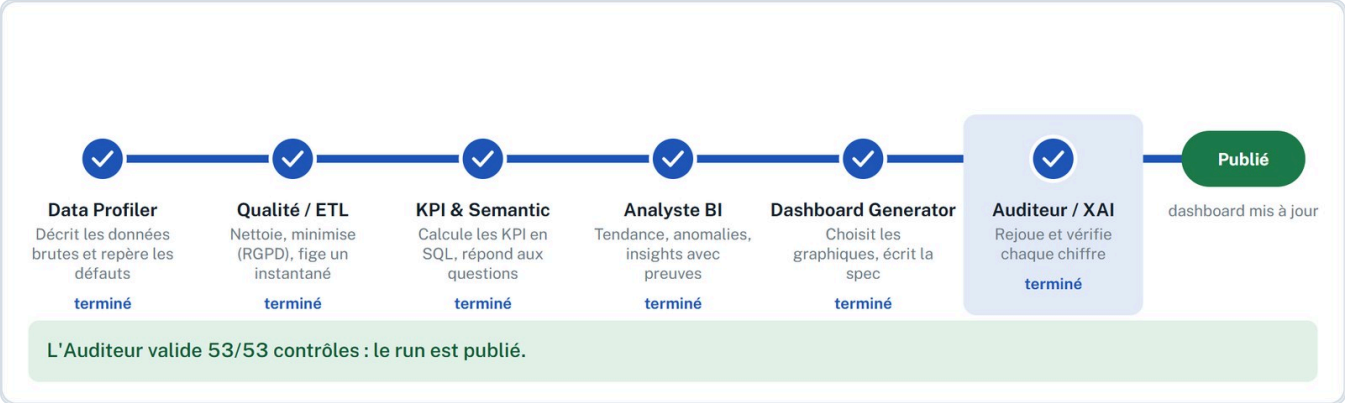
1. Le clic envoie la demande à l'API, qui lance le vrai pipeline LangGraph en arrière-plan.
2. L'orchestrateur appelle les 6 agents dans l'ordre ; après chacun, il **valide** sa sortie.
3. Chaque essai est écrit dans la trace ; l'écran interroge l'API 3 fois par seconde et allume la station.
4. KPI & Semantic et l'Analyste appellent le LLM local (questions, rédaction) : ce sont les étapes les plus longues.
5. L'Auditeur rejoue tout : 53 contrôles.
6. Aucun écart : l'orchestrateur **publie** (le pointeur `published.json` change), le tableau de bord passe sur ce run.

Comment le montrer

1. Page **Scénarios** : *Run nominal* est déjà choisi ; « Ralentir pour la présentation » coché.
2. Cliquer **Lancer le scénario**.
3. Pointer les stations qui s'allument une par une, et le journal en dessous.
4. À la fin : « L'Auditeur valide 53/53 contrôles : le run est publié ».
5. Cliquer l'onglet **1 Data Profiler** : les 3 défauts.

La phrase-résultat

> « Les 6 agents ont travaillé sur 265 609 demandes réelles ; chaque chiffre a été vérifié, le run est publié. »



Fin du scénario 1 : toutes les stations terminées, « Publié ».

2 Panne d'un agent

Agent 2, Qualité / ETL. Page Scénarios. Environ 20 s. Résultat : Échec, rien n'est publié

Comment ça marche

1. L'agent choisi est remplacé par une version qui **lève une erreur à chaque appel** (option `--simulate-failure`).
2. Le Data Profiler passe normalement.
3. Qualité / ETL échoue : l'orchestrateur attend 1 s et relance (essai 2), puis encore (essai 3).
4. Au 3e échec : statut **failed** ; les 4 agents suivants ne sont jamais appelés.
5. Pas de publication : le tableau de bord ne change pas.

Comment le montrer

1. Cocher *Panne d'un agent* ; « Agent en panne » : **Qualité / ETL**.
2. **Lancer le scénario**.
3. Lire le journal en orange : « Essai 1 échoué… l'orchestrateur relance l'étape ».
4. Pointer la station rouge (« échec après 3 essais ») et la fin de ligne « Bloqué ».

La phrase-résultat

> « Une panne ne produit jamais un tableau de bord partiel : rien n'est publié. »



Fin du scénario 2 : Qualité / ETL en échec après 3 essais, les stations suivantes en attente, « Bloqué ».

3 Question métier et refus

Agent 3, KPI & Semantic. Page Poser une question. Environ 1 min. Résultat : 15 443, puis un refus motivé

Comment ça marche

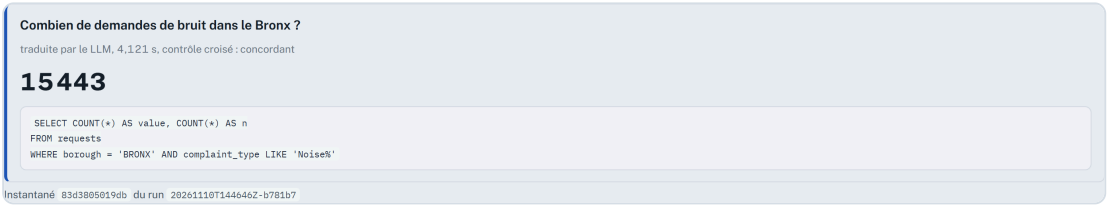
1. La question est **nettoyée** : e-mail, téléphone, adresse, n° de demande masqués.
2. **Contrôle de périmètre** : une cause (« pourquoi »), des moyens (« agents », « budget ») ou une prévision sont refusés **tout de suite**, sans requête ni LLM.
3. Sinon, le LLM reçoit la question, le schéma et les définitions, et renvoie du SQL.
4. DuckDB exécute le SQL dans un **bac à sable** (lecture seule, SELECT seul), sur l'instantané audité.
5. Résultat sur moins de 10 demandes : masqué.
6. **Contrôle croisé** avec les règles : « concordant » ou « à vérifier ».

Comment le montrer

1. Page **Poser une question**.
2. Taper « Combien de demandes de bruit dans le Bronx ? », **Répondre** (2 à 5 s).
3. Montrer 15 443, le SQL, « traduite par le LLM, contrôle croisé : concordant ».
4. Cliquer l'exemple « Combien d'agents faut-il recruter ? » : refus avec la raison, en 0,1 s.

La phrase-résultat

> « Le LLM traduit, le code calcule ; et quand les données ne permettent pas de répondre, on refuse. »



Scénario 3 : la réponse 15 443 avec le SQL écrit par le LLM et le contrôle croisé.

4 Détection d'anomalie

Agent 4, Analyste BI. Page Tableau de bord. Environ 1 min. Résultat : 460 demandes au lieu d'environ 218

Comment ça marche

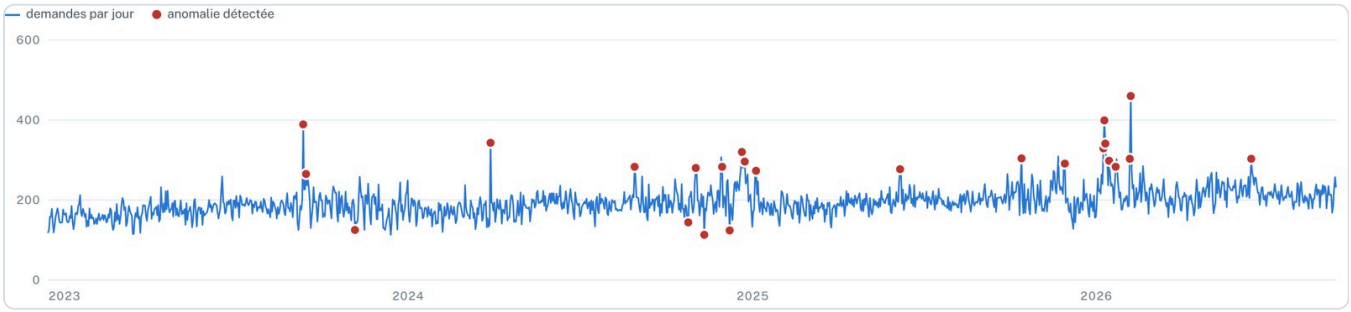
1. L'Analyste prend le nombre de demandes par jour (1 369 jours).
2. **STL** sépare la tendance, le rythme de la semaine et le reste.
3. Sur le reste, un **z-score robuste** (médiane et MAD) : un jour au-delà de 5 est anormal.
4. Pour ce jour : le type de plainte le plus en hausse par rapport aux 28 jours d'avant, écrit comme une **piste**.
5. Le LLM rédige le texte ; chaque nombre doit être dans les preuves, sinon le texte est rejeté.
6. Le Dashboard Generator dessine la courbe avec les points rouges.

Comment le montrer

1. Page **Tableau de bord**, descendre à « Demandes par jour ».
2. Survoler la courbe jusqu'au plus haut point rouge : **24 févr. 2026, 460 demandes**.
3. Plus bas, « Pics inhabituels » : l'insight du 24 février, *Snow or Ice*, « piste à vérifier, pas une cause ».
4. Cliquer **Voir les preuves**.

La phrase-résultat

> « Le système trouve le jour anormal et une piste, mais c'est à l'humain de conclure. »



Scénario 4 : les points rouges sont les jours anormaux ; le plus haut est le 24 février 2026.

5 Écart persistant

Agent 5, Dashboard Generator. Page Scénarios. Environ 1 min 40. Résultat : **Non fiable**, rien n'est publié

Comment ça marche

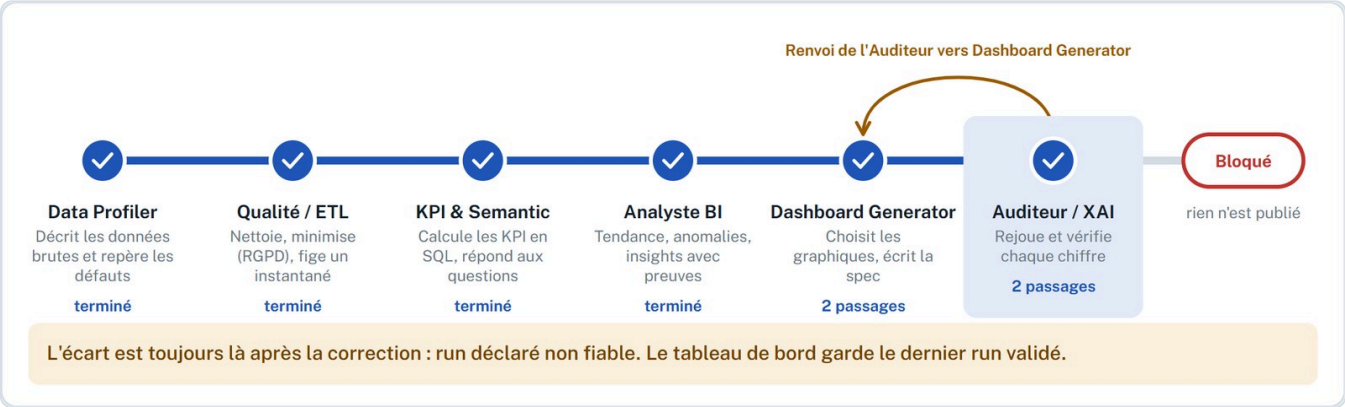
1. Les 6 agents tournent normalement.
2. L'Auditeur ajoute un écart bloquant sur l'étape choisie (option `--simulate-audit-flag … --persistent`), en plus de ses vrais contrôles.
3. L'orchestrateur **renvoie** le run vers cette étape avec le message de correction ; elle et les suivantes refont leur travail.
4. Second audit : l'écart est **toujours là** ; un seul renvoi est autorisé.
5. Statut **unreliable** (non fiable) : pas de publication, le pointeur garde le dernier run validé.

Comment le montrer

1. Décocher « Ralentir » (plus rapide).
2. Cocher *Écart persistant* ; « Étape signalée » : **Dashboard Generator** ; **Lancer**.
3. Pendant le run : présenter son agent (onglet du run nominal ou ce guide).
4. Montrer la flèche « Renvoi de l'Auditeur vers Dashboard Generator », puis « non fiable ».
5. Page **Tableau de bord** : inchangé. Page **Historique** : « Non fiable ».

La phrase-résultat

> « Si l'erreur reste après correction, on ne publie rien : le tableau de bord garde le dernier bon run. »



Fin du scénario 5 : flèche de renvoi, « 2 passages », ligne « Bloqué ».

6 Écart corrigé

Agent 6, Auditeur / XAI. Page Scénarios. Environ 2 min 30. Résultat : Publié après un second audit

Comment ça marche

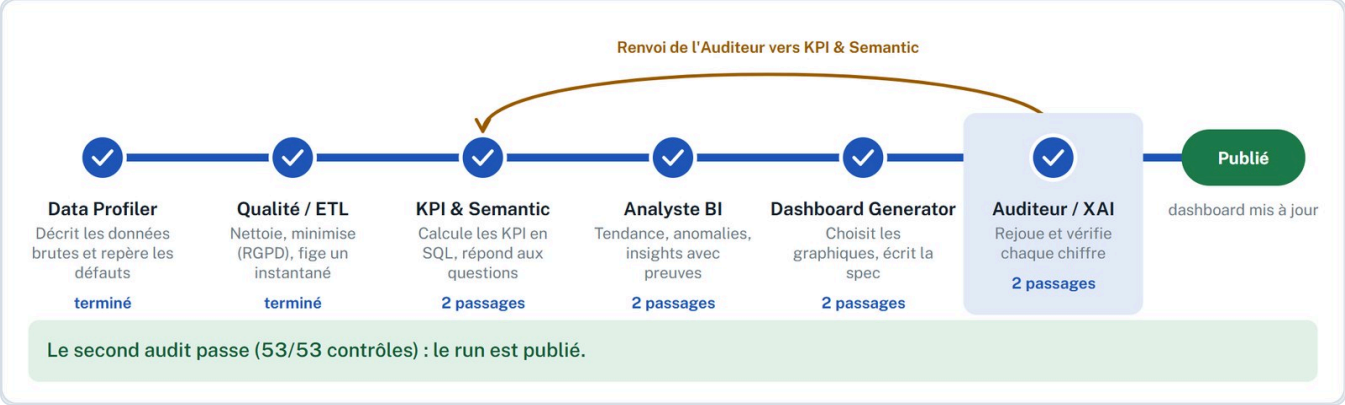
1. Les 6 agents tournent normalement.
2. Au premier audit, l'Auditeur signale un écart sur KPI & Semantic (option `--simulate-audit-flag kpi_semantic`).
3. Renvoi : KPI, Analyste, Dashboard et Auditeur refont leur travail, avec le message de correction.
4. Second audit : plus d'écart, **53/53**.
5. Statut **success** : publication ; le renvoi reste visible dans le journal et la trace.

Comment le montrer

1. Décocher « Ralentir ».
2. Cocher *Écart corrigé par l'Auditeur* ; « Étape signalée » : **KPI & Semantic** ; **Lancer**.
3. Pendant le run : expliquer les 53 contrôles de l'Auditeur.
4. Montrer la flèche, « 2 passages » sous les stations 3 à 6, puis « Le second audit passe (53/53) : le run est publié ».
5. Onglet **6 Auditeur / XAI** : les deux passages d'audit.

La phrase-résultat

> « L'Auditeur a trouvé l'erreur, l'étape fautive l'a corrigée, et seulement ensuite on publie. »



Fin du scénario 6 : renvoi vers KPI & Semantic, second audit réussi, « Publié ».

1 Data Profiler

Présenté par _____

Je décris les données brutes et je repère tout ce qui ne va pas, avant la moindre transformation.

Mon scénario **1. Run nominal** page Scénarios, environ 1 min 30, Publié

Déclencheur	début du run
Entrée	l'extrait brut : 265 609 demandes, 12 colonnes
Sortie	nombre de lignes, valeurs nulles, plage de dates, répartitions, défauts typés par colonne
LLM	aucun : détection déterministe et reproductible
Code	`src/biflow/agents/data_profiler.py`

Le scénario, étape par étape

1. L'orchestrateur m'appelle avec la référence de l'extrait.
2. Je vérifie que les colonnes obligatoires sont là.
3. Je convertis dates et nombres ; une valeur non convertible compte comme « mauvais type ».
4. Je calcule valeurs nulles, plage de dates, répartitions par type, borough et statut.
5. Je cherche 7 familles de défauts (doublon, mauvais type, manquant, incohérence, format, valeur invalide, hors plage), chacune avec un compteur et des exemples.
6. Je lis la fiche de provenance : ici, un échantillon d'environ 2 %, non probabiliste.
7. L'orchestrateur valide ma sortie (colonnes, au moins 1 ligne, taux entre 0 et 1).

Les chiffres à retenir

265 609
demandes, du 1er janvier 2023 au 30 septembre 2026

3
types de défaut trouvés et transmis à Qualité / ETL

139
clôtures avant la création (incohérence)

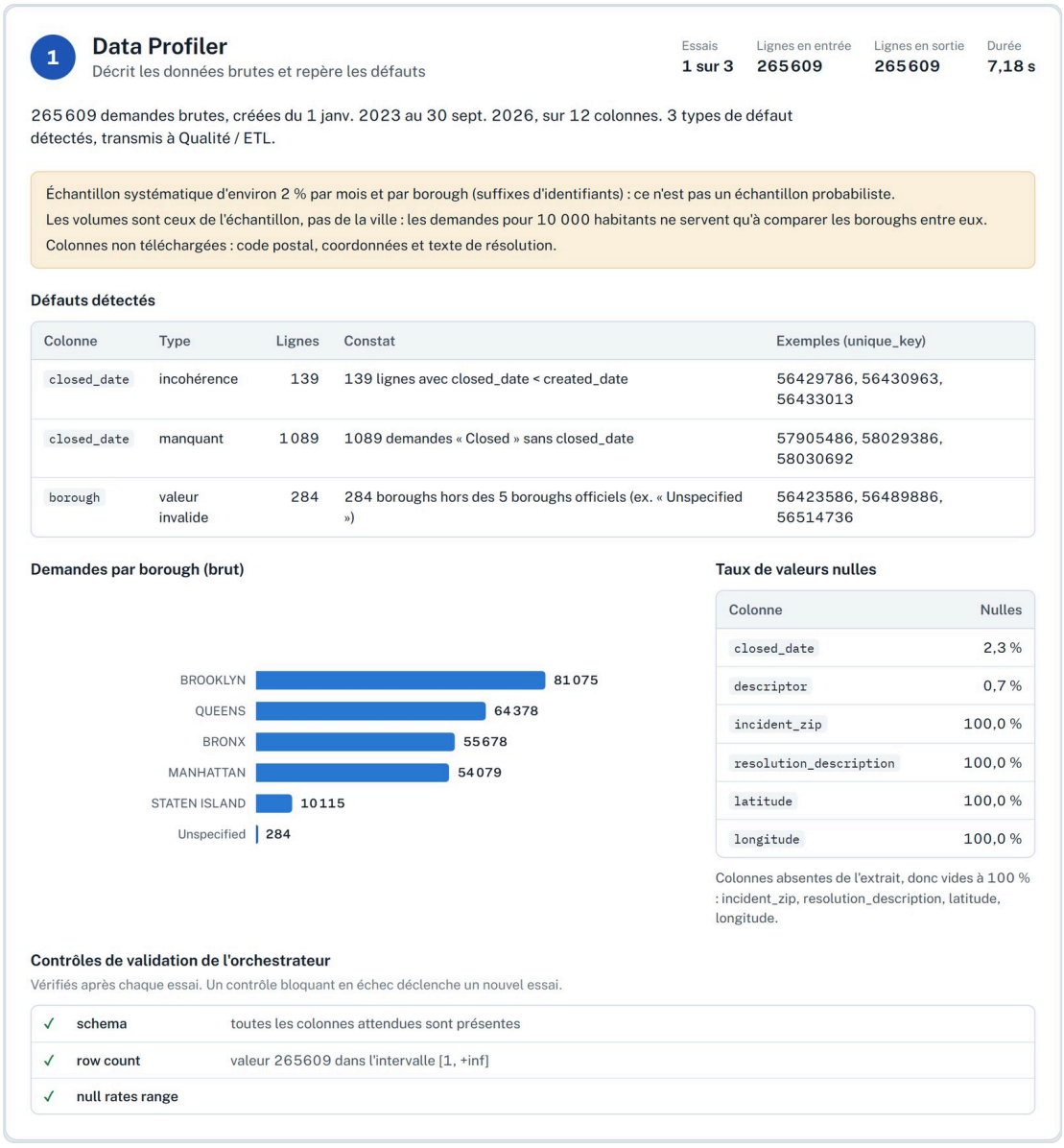
1 089
demandes « Closed » sans date de clôture

284
boroughs hors des 5 officiels (« Unspecified »)

9 / 9
types de défauts injectés retrouvés à l'évaluation

Ce que je montre

1. Sous le schéma de ligne, onglet **1 Data Profiler**.
2. Le bandeau orange : la réserve sur l'échantillon.
3. Le tableau **Défauts détectés** : colonne, type, nombre, exemples.
4. Les barres par borough et les taux de valeurs nulles.
5. En bas : les contrôles de validation de l'orchestrateur.



Panneau Data Profiler du run nominal.

Ce que je dis (environ 2 minutes)

> « Je suis le premier agent : le Data Profiler. Mon rôle est de dire à quoi ressemblent les données **avant** qu'on les touche. Je ne corrige rien : je constate, et je passe la liste à l'agent suivant.
>
> Sur l'extrait réel, 265 609 demandes NYC 311 de 2023 à 2026, j'ai trouvé trois types de défaut. 139 demandes ont une date de clôture **avant** leur date de création : c'est impossible, donc incohérent. 1 089 demandes sont marquées « Closed » mais sans date de clôture. Et 284 demandes ont un borough qui n'est pas un des cinq officiels, par exemple « Unspecified ».
>
> Chaque défaut est rattaché à une colonne, avec un compteur et des numéros de demande en exemple : n'importe qui peut aller vérifier. Je lis aussi la fiche de provenance de l'extrait, qui dit que c'est un échantillon d'environ 2 % : cette réserve est reprise jusqu'au tableau de bord.
>
> Je n'utilise pas de LLM : une détection doit être reproductible. À l'évaluation, sur des données où nous avions injecté 9 types de défauts, je les ai tous retrouvés. Je passe maintenant la main à Qualité / ETL, qui va décider quoi faire de ces défauts. »

Quand ça tourne mal

Cas	Ce qui se passe
Une colonne obligatoire manque	erreur, 3 essais, puis run Échec : rien n'est publié
Extrait vide	la validation « au moins 1 ligne » échoue : même chemin, échec
Colonne facultative absente	non bloquant : listée comme absente (ici code postal, coordonnées, texte de résolution)
Scénario *Panne d'un agent* sur Data Profiler	3 essais visibles dans le journal, la ligne s'arrête dès la première station

Questions possibles du jury

Pourquoi le Profiler ne corrige-t-il pas lui-même ?
Pour séparer constat et décision : chaque correction est une règle tracée de l'ETL, qu'on peut relire et contester.

Comment savez-vous qu'il trouve bien les défauts ?
Par l'évaluation sur un jeu synthétique où les défauts sont connus d'avance : 9 types injectés, 9 détectés.

Pourquoi 100 % de valeurs nulles sur certaines colonnes ?
Ces colonnes n'ont pas été téléchargées dans l'extrait (code postal, coordonnées, texte libre) ; le Profiler le signale au lieu de l'ignorer.

2 Qualité / ETL

Présenté par _____

Je produis une couche de données propre, protégée et figée, sans jamais perdre une ligne sans trace.

Mon scénario **2. Panne d'un agent** — page Scénarios, environ 20 s, Échec

Déclencheur	sortie du Profiler validée
Entrées	données brutes, profil, délais cibles par type de plainte
Sortie	instantané parquet et son empreinte SHA-256, règles appliquées, quarantaine
LLM	aucun
Code	`src/biflow/agents/quality_etl.py`

Le scénario, étape par étape

1. **Supprimer** les lignes inutilisables (clé absente, date invalide, doublon, clôture avant création, type absent), chacune copiée en **quarantaine** avec sa règle.
2. **Corriger** sans supprimer : borough inconnu → `UNSPECIFIED`, code postal ou coordonnées invalides → vide.
3. **Protéger** (RGPD) : texte libre retiré, coordonnées arrondies à environ 1 km.
4. **Enrichir** : délai de résolution en heures, délai cible, « sous le délai cible ».
5. **Figer** l'instantané et calculer son empreinte : tous les agents suivants lisent ce fichier et lui seul.
6. Validation : lues = conservées + supprimées, quarantaine = supprimées, aucun texte libre.

Les chiffres à retenir

265 609
lignes lues

265 470
lignes conservées dans l'instantané

139
supprimées (clôture avant création), toutes en quarantaine

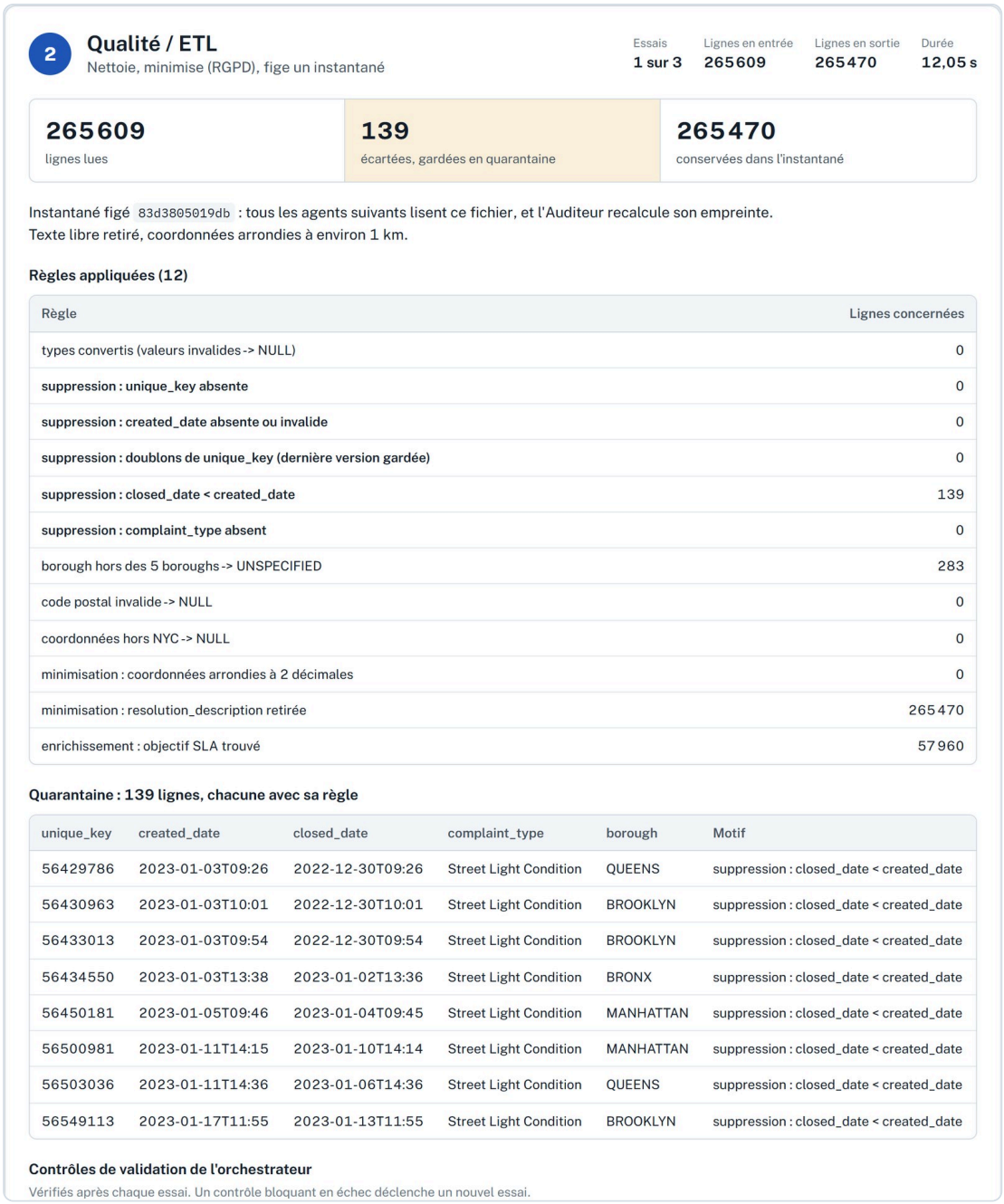
12
règles tracées, avec le nombre de lignes concernées

283
boroughs ramenés à `UNSPECIFIED`

83d380...
empreinte de l'instantané, revérifiée par l'Auditeur

Ce que je montre

1. Onglet **2 Qualité / ETL** : les trois compteurs (lues, écartées, conservées).
2. Le tableau **Règles appliquées** : 12 règles et leurs lignes.
3. La **Quarantaine** : les 139 lignes, avec leur motif.
4. Les contrôles : « rows balance », « dropped rows quarantined », « no free text ».
5. Puis le scénario **Panne d'un agent** sur Qualité / ETL (20 s).



2 Qualité / ETL

Nettoie, minimise (RGPD), fige un instantané

Essais	Lignes en entrée	Lignes en sortie	Durée
1 sur 3	265609	265470	12,05 s

265609	139	265470
lignes lues	écartées, gardées en quarantaine	conservées dans l'instantané

Instantané figé `83d3805019db` : tous les agents suivants lisent ce fichier, et l'Auditeur recalcule son empreinte.
Texte libre retiré, coordonnées arrondies à environ 1 km.

Règles appliquées (12)

Règle	Lignes concernées
types convertis (valeurs invalides -> NULL)	0
suppression : unique_key absente	0
suppression : created_date absente ou invalide	0
suppression : doublons de unique_key (dernière version gardée)	0
suppression : closed_date < created_date	139
suppression : complaint_type absent	0
borough hors des 5 boroughs -> UNSPECIFIED	283
code postal invalide -> NULL	0
coordonnées hors NYC -> NULL	0
minimisation : coordonnées arrondies à 2 décimales	0
minimisation : resolution_description retirée	265470
enrichissement : objectif SLA trouvé	57960

Quarantaine : 139 lignes, chacune avec sa règle

unique_key	created_date	closed_date	complaint_type	borough	Motif
56429786	2023-01-03T09:26	2022-12-30T09:26	Street Light Condition	QUEENS	suppression : closed_date < created_date
56430963	2023-01-03T10:01	2022-12-30T10:01	Street Light Condition	BROOKLYN	suppression : closed_date < created_date
56433013	2023-01-03T09:54	2022-12-30T09:54	Street Light Condition	BROOKLYN	suppression : closed_date < created_date
56434550	2023-01-03T13:38	2023-01-02T13:36	Street Light Condition	BRONX	suppression : closed_date < created_date
56450181	2023-01-05T09:46	2023-01-04T09:45	Street Light Condition	MANHATTAN	suppression : closed_date < created_date
56500981	2023-01-11T14:15	2023-01-10T14:14	Street Light Condition	MANHATTAN	suppression : closed_date < created_date
56503036	2023-01-11T14:36	2023-01-06T14:36	Street Light Condition	QUEENS	suppression : closed_date < created_date
56549113	2023-01-17T11:55	2023-01-13T11:55	Street Light Condition	BROOKLYN	suppression : closed_date < created_date

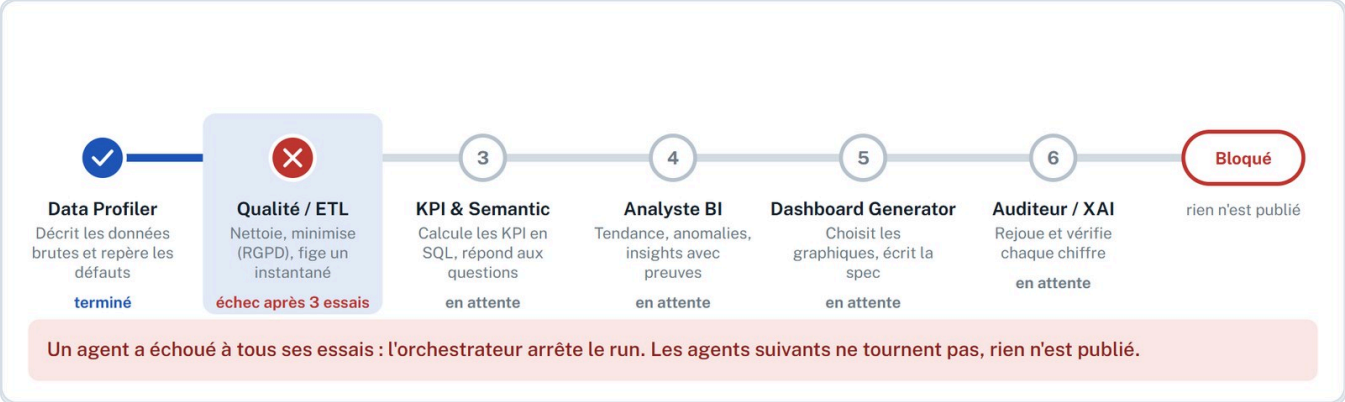
Contrôles de validation de l'orchestrateur

Vérifiés après chaque essai. Un contrôle bloquant en échec déclenche un nouvel essai.

Panneau Qualité / ETL : compteurs, règles appliquées et début de la quarantaine.

Ce que je dis (environ 2 minutes)

> « Je suis l'agent Qualité / ETL. Je reçois les données brutes et la liste des défauts du Profiler, et je produis la couche propre que tous les autres agents vont utiliser.
>
> Ma règle principale : **aucune ligne ne disparaît sans trace**. Sur 265 609 demandes, j'en ai supprimé 139, celles qui sont clôturées avant d'être créées. Elles ne sont pas perdues : elles sont en quarantaine avec leur motif, ici. Et l'orchestrateur vérifie que lues = conservées + supprimées.
>
> Je ne supprime que ce qui est inutilisable. Le reste, je le corrige : 283 boroughs inconnus deviennent « UNSPECIFIED ». Chaque action est une des 12 règles de ce tableau, avec le nombre de lignes touchées.
>
> Je protège aussi les données personnelles : le texte libre est retiré et les coordonnées arrondies à environ 1 km. Enfin, je fige le résultat dans un instantané avec une empreinte SHA-256 : si une seule valeur change, l'empreinte change, et l'Auditeur le verra.
>
> Pour montrer la robustesse, je lance le scénario « Panne d'un agent » sur mon étape. [Lancer.] L'orchestrateur me relance deux fois, puis il arrête tout : les agents suivants ne tournent pas, et rien n'est publié. Une panne ne produit jamais un tableau de bord partiel. »



Scénario Panne d'un agent sur Qualité / ETL : échec après 3 essais, la ligne s'arrête sur « Bloqué ».

Quand ça tourne mal

Cas	Ce qui se passe
Toutes les lignes seraient supprimées	la validation « au moins une ligne conservée » échoue : 3 essais, puis Échec
Une ligne disparaît sans quarantaine	la validation « supprimées = quarantaine » échoue : même chemin
L'Auditeur signale l'étape	relancée une fois avec la correction demandée, notée dans la trace

Questions possibles du jury

Pourquoi garder les 1 089 demandes « Closed » sans date ?
La demande existe bien : on la compte dans les volumes ; seul son délai n'est pas calculable, donc elle n'entre pas dans le délai médian.

À quoi sert l'empreinte ?
À prouver que tous les agents et l'Auditeur ont travaillé sur exactement les mêmes données.

La panne est-elle réelle ?
Simulée par une option, comme en ligne de commande ; les relances et l'arrêt sont le vrai comportement de l'orchestrateur.

3 KPI & Semantic

Présenté par _____

Je calcule les indicateurs en SQL à partir de définitions communes, et je réponds aux questions métier sans laisser le LLM produire un chiffre.

Mon scénario **3. Question métier et refus** — page Poser une question, environ 1 min

Déclencheur	sortie Qualité / ETL validée
Entrées	l'instantané, le catalogue `kpi_catalog.yaml` (relu dans Git), les questions
Sortie	KPI, ventilations, réponses, journal de toutes les requêtes
LLM	traduit une question en SQL ; le SQL est exécuté par le code
Code	`src/biflow/agents/kpi_semantic.py`, `src/biflow/semantic/`

Scénario A : les KPI du catalogue

1. Charger l'instantané dans un **bac à sable DuckDB** : lecture seule, tables autorisées, temps et lignes limités.
2. Exécuter les 5 KPI et les 6 ventilations du catalogue.
3. **Masquer** tout résultat sur moins de 10 demandes.
4. Enregistrer chaque requête : SQL, empreinte des données, résultat.

Scénario B : une question métier

1. Masquer e-mail, téléphone, adresse avant tout envoi.
2. Vérifier le périmètre : une cause, des effectifs ou une prévision sont **refusés**.
3. Le LLM propose du SQL, exécuté dans le bac à sable.
4. **Contrôle croisé** avec la baseline à règles : « concordant » ou « à vérifier ».

Les chiffres à retenir

97,7 %
des demandes sont clôturées

7,3 h
délai médian de résolution

93,5 %
sous le délai cible (cibles provisoires)

301,2
demandes pour 10 000 habitants

15 443
demandes de bruit dans le Bronx (LLM, concordant)

88 %
des questions bien traduites en SQL avec le LLM, contre 61 % sans

Ce que je montre

1. Onglet **3 KPI & Semantic** : les 5 KPI, « Voir le SQL » sous l'un d'eux.
2. **Questions métier** : la réponse 15 443, le SQL écrit par le LLM, « contrôle croisé : concordant ».
3. La question « Pourquoi… » : **refusée**, avec la raison.
4. **Scénario 3**, page **Poser une question** : « Combien de demandes de bruit dans le Bronx ? » → 15 443 ; puis « Combien d'agents faut-il recruter ? » → refus en direct.



Panneau KPI & Semantic : les 5 KPI avec leur SQL, puis les réponses aux questions.

Ce que je dis (environ 2 minutes)

> « Je suis l'agent KPI & Semantic. Je calcule les indicateurs, et je réponds aux questions métier.
>
> Les indicateurs ne sont pas inventés à chaque run : ils sont définis une fois dans un catalogue, relu dans Git, et exécutés en SQL. Résultat : 97,7 % des demandes clôturées, un délai médian de 7,3 heures, 93,5 % sous le délai cible. Sous chaque chiffre, « Voir le SQL » montre la requête exacte et l'empreinte des données utilisées.
>
> Pour les questions, c'est là qu'intervient le LLM, mais seulement pour **traduire**. « Combien de demandes de bruit dans le Bronx ? » : le LLM écrit ce SQL, et c'est DuckDB qui l'exécute, en lecture seule, dans un bac à sable. Résultat : 15 443. Et nous comparons avec notre baseline à règles : ici, les deux concordent.
>
> Je sais aussi dire non. « Pourquoi les délais sont-ils plus longs à Staten Island ? » : les données montrent des délais, pas leurs causes, donc je refuse, sans appeler le LLM. Même chose pour « Combien d'agents faut-il recruter ? ». [Montrer en direct.]
>
> Enfin, aucun résultat sur moins de 10 demandes n'est affiché, pour qu'on ne puisse pas identifier une personne. »

Quand ça tourne mal

Cas	Ce qui se passe
Le LLM écrit un SQL interdit (UPDATE, table non autorisée) ou faux	refusé par le bac à sable ; l'erreur est renvoyée une fois au LLM, sinon la baseline prend le relais
Question hors périmètre (cause, effectifs, prévision)	refus motivé, sans requête ni LLM ; le run continue
Question avec une adresse ou un e-mail	masqués avant l'envoi, listés comme données personnelles trouvées
L'Auditeur trouve un KPI différent de sa requête	renvoi vers cet agent : c'est le scénario 6 *Écart corrigé*, présenté par l'agent 6

Questions possibles du jury

Le LLM peut-il inventer un chiffre ?
Non : il écrit du SQL, le chiffre vient de l'exécution. Les KPI du tableau de bord viennent tous du catalogue, sans LLM.

Et si le SQL du LLM est dangereux ?
Bac à sable DuckDB : copie en mémoire, SELECT seul, tables autorisées, fichiers coupés, temps et lignes limités.

Qu'apporte le LLM, concrètement ?
Les reformulations libres : 76 % justes avec le LLM contre 5 % pour les règles seules ; 88 % sur l'ensemble des 51 questions.

Les délais cibles sont-ils officiels ?
Non, provisoires ; le KPI l'écrit en toutes lettres, ils restent à valider par le service.

4 Analyste BI

Présenté par _____

Je transforme les KPI en constats lisibles, chacun avec ses preuves chiffrées, et je détecte les jours anormaux.

Mon scénario **4. Détection d'anomalie** — page Tableau de bord, environ 1 min

Déclencheur	sortie KPI validée
Entrées	KPI et ventilations, instantané, provenance de l'extrait
Sortie	insights (titre, texte, preuves), anomalies, tendance
LLM	réécrit les textes pour des non-techniciens ; chaque nombre est vérifié
Code	`src/biflow/agents/analyste_bi.py`, `src/biflow/analysis/`

Le scénario, étape par étape

1. Faits de synthèse : volume, délais, délai cible, demandes par habitant.
2. Tendance : dernière année comparée à la précédente.
3. Anomalies : décomposition **STL** (tendance, saisonnalité, reste) puis **z-score robuste** sur le reste, pour la ville et chaque borough.
4. Pour chaque pic : le type de plainte le plus en hausse, présenté comme « piste à vérifier, pas une cause ».
5. Chaque insight porte ses **preuves** (les nombres calculés).
6. Le LLM réécrit ; **tout nombre absent des preuves fait rejeter le texte**, le gabarit est gardé.

Les chiffres à retenir

10
insights, tous avec leurs preuves

+9,8 %
volume journalier, dernière année contre la précédente

24
jours anormaux pour toute la ville (61 avec les boroughs)

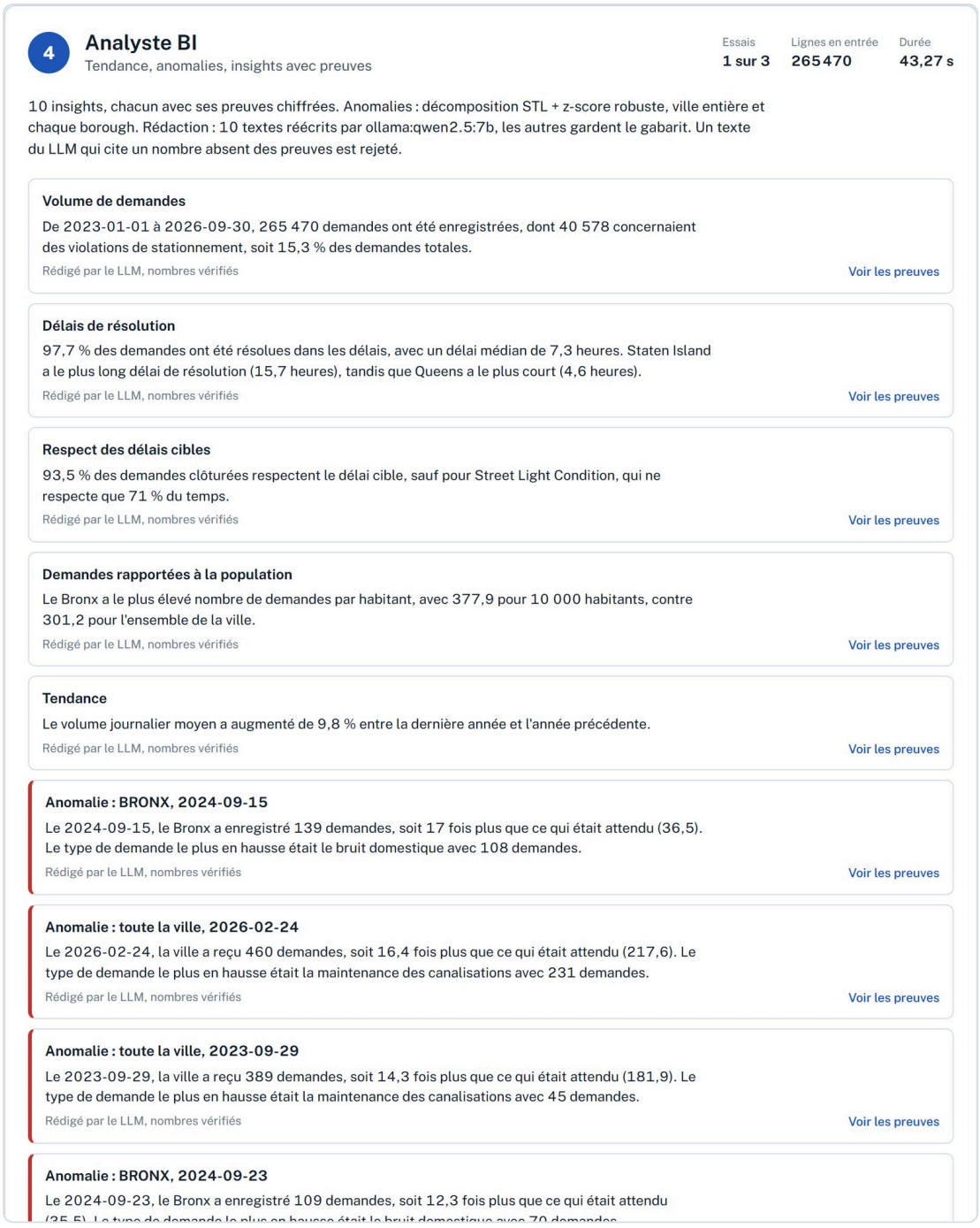
460
demandes le 24/02/2026 au lieu d'environ 218 : *Snow or Ice*

389
le 29/09/2023 au lieu d'environ 182 : jour des inondations

10 / 10
textes du LLM acceptés, 0 nombre non justifié

Ce que je montre

1. Onglet **4 Analyste BI** : les insights, « Rédigé par le LLM, nombres vérifiés ».
2. « Voir les preuves » sous un insight.
3. Le tableau des anomalies : observé, attendu, écart.
4. **Scénario 4**, page **Tableau de bord** : survoler la courbe jusqu'au 24 février 2026 (460 demandes), puis l'insight « Anomalie : toute la ville, 2026-02-24 » et ses preuves.



Panneau Analyste BI : insights rédigés par le LLM, avec leurs preuves.

Ce que je dis (environ 2 minutes)

> « Je suis l'Analyste BI. Les KPI sont des nombres ; mon rôle est d'en tirer des constats lisibles, et de repérer ce qui sort de l'ordinaire.
>
> D'abord la tendance : le volume journalier a augmenté de 9,8 % entre la dernière année et la précédente. Ensuite les anomalies : je décompose la série jour par jour en tendance, saisonnalité et reste, et je cherche les jours où le reste est extrême. Sur toute la ville, 24 jours ressortent.
>
> Le plus fort : le 24 février 2026, 460 demandes au lieu d'environ 218. Ce jour-là, le type qui explose est « Snow or Ice ». Et le 29 septembre 2023, c'est « Sewer Maintenance », le jour des inondations à New York. Mais je l'écris comme **une piste à vérifier, pas une cause** : c'est à l'analyste humain de conclure.
>
> Le LLM réécrit mes textes pour qu'ils soient lisibles par un responsable. Mais chaque nombre qu'il écrit doit exister dans mes preuves calculées ; sinon, son texte est rejeté et je garde mon gabarit. Sur ce run : 10 textes, 10 acceptés, aucun chiffre inventé. »

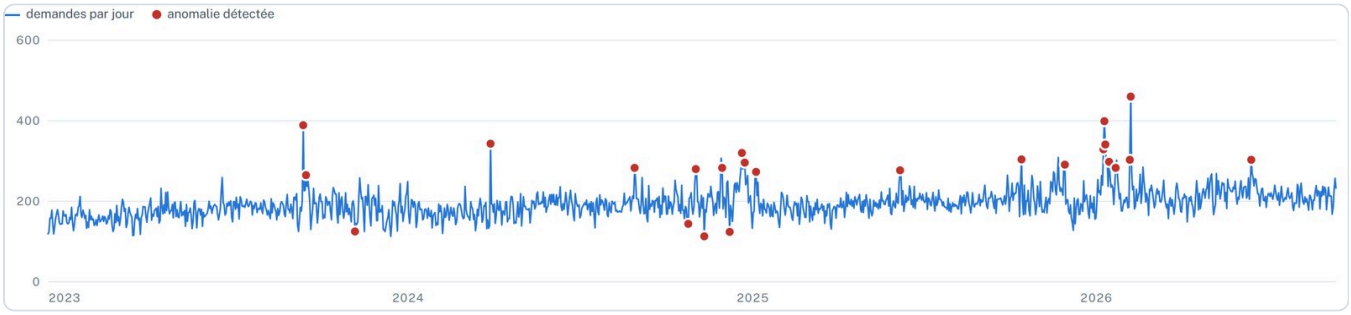


Tableau de bord, demandes par jour : les points rouges sont les anomalies détectées.

Quand ça tourne mal

Cas	Ce qui se passe
Le LLM écrit « environ 500 » alors que la preuve dit 460	texte rejeté, gabarit gardé, compté dans les notes (« 1 rejeté »)
LLM indisponible ou réponse illisible	tous les gabarits sont gardés, le run continue
Période de moins de 7 jours	pas de détection d'anomalies, un insight l'indique
Borough trop petit	pas d'analyse d'anomalies sur ce borough (seuil de volume)

Questions possibles du jury

Pourquoi ne pas donner la cause ?
Les données contiennent des volumes et des délais, pas les causes ; affirmer une cause serait inventer.

Combien de fausses alertes ?
Sur le jeu d'évaluation : l'anomalie injectée trouvée (bon jour, bonne zone), 0 faux positif. Le seuil a été ajusté après les tests réels.

Pourquoi STL ?
Les demandes ont une saisonnalité hebdomadaire et annuelle : STL l'enlève avant de chercher les pics, sinon chaque lundi serait une alerte.

5 Dashboard Generator

Présenté par _____

Je décide quoi afficher et comment, dans une spécification vérifiable. Je ne publie rien moi-même.

Mon scénario **5. Écart persistant** — page Scénarios, environ 1 min 40, Non fiable

Déclencheur	sortie Analyste validée
Entrées	KPI, ventilations, réponses, insights, anomalies
Sortie	`dashboard_spec.json` et son empreinte ; chaque graphique avec ses requêtes d'origine
LLM	aucun : choix par règles, reproductibles
Code	`src/biflow/agents/dashboard_generator.py`

Les règles de choix

Donnée	Graphique
KPI global	carte KPI
série par jour	courbe, anomalies marquées
6 modalités ou moins	barres verticales
plus de 6 modalités	barres horizontales, 15 premières
réponses aux questions	table
insights	bloc de texte

Les chiffres à retenir

13
éléments dans la spécification

5
cartes KPI

5
graphiques en barres (dont 1 horizontal)

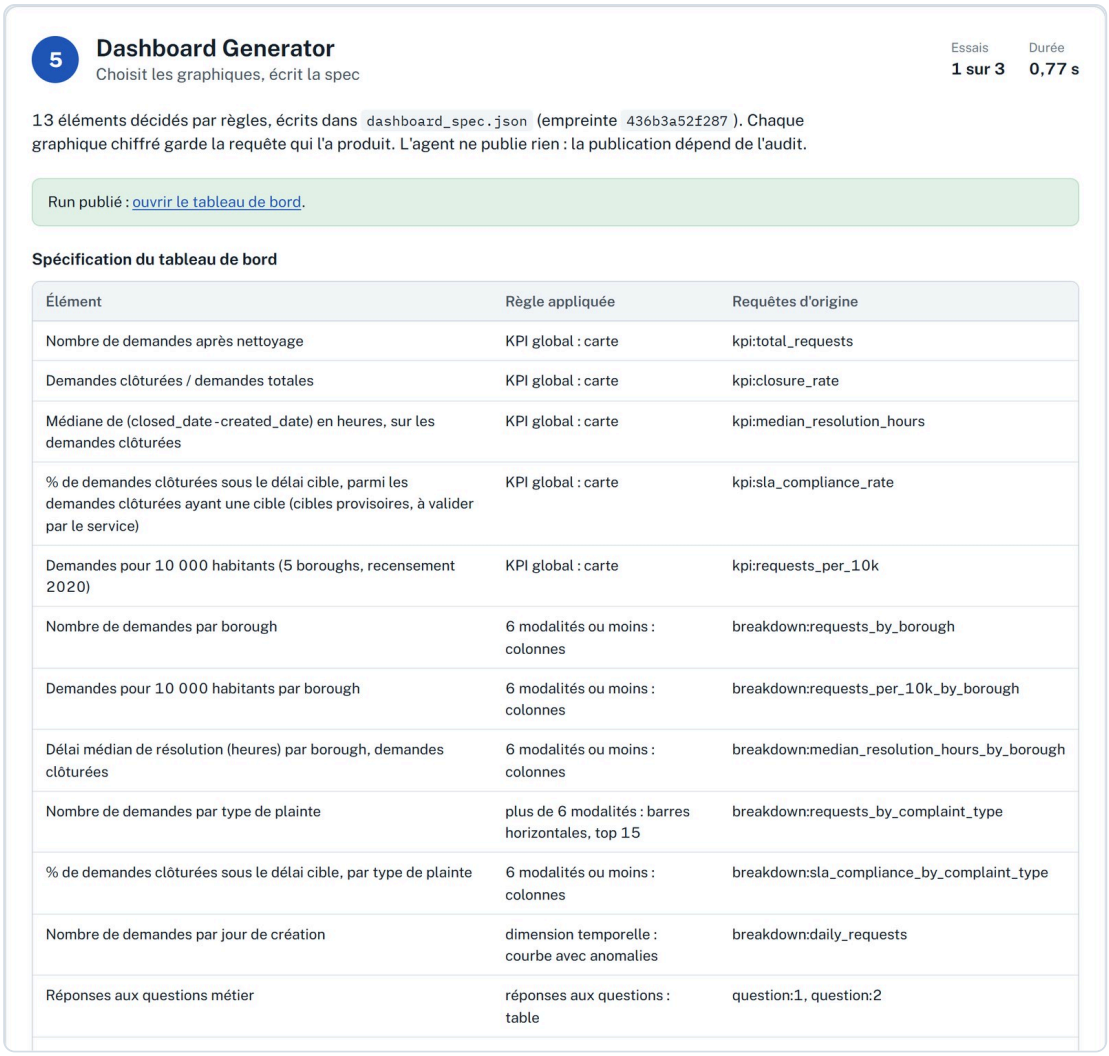
1 + 1 + 1
courbe, table des questions, bloc d'insights

0
graphique chiffré sans requête d'origine

12 / 12
contrôles de l'Auditeur sur le tableau de bord

Ce que je montre

1. Onglet **5 Dashboard Generator** : le tableau de la spécification (élément, règle appliquée, requêtes d'origine).
2. Page **Tableau de bord** : « Spec vérifiée » et son empreinte, en haut à droite.
3. « Voir le SQL » sous une carte : la requête d'origine.
4. **Scénario 5** : *Écart persistant*, étape signalée **Dashboard Generator**, **Lancer**. La flèche de renvoi, puis « non fiable ».
5. Retour sur **Tableau de bord** : il n'a pas changé ; **Historique** : le run « Non fiable ».



5 **Dashboard Generator**
Choisit les graphiques, écrit la spec

Essais 1 sur 3 | Durée 0,77 s

13 éléments décidés par règles, écrits dans `dashboard_spec.json` (empreinte `436b3a52f287`). Chaque graphique chiffré garde la requête qui l'a produit. L'agent ne publie rien : la publication dépend de l'audit.

Run publié : ouvrir le tableau de bord.

Spécification du tableau de bord

Élément	Règle appliquée	Requêtes d'origine
Nombre de demandes après nettoyage	KPI global : carte	kpi:total_requests
Demandes clôturées / demandes totales	KPI global : carte	kpi:closure_rate
Médiane de (closed_date - created_date) en heures, sur les demandes clôturées	KPI global : carte	kpi:median_resolution_hours
% de demandes clôturées sous le délai cible, parmi les demandes clôturées ayant une cible (cibles provisoires, à valider par le service)	KPI global : carte	kpi:sla_compliance_rate
Demandes pour 10 000 habitants (5 boroughs, recensement 2020)	KPI global : carte	kpi:requests_per_10k
Nombre de demandes par borough	6 modalités ou moins : colonnes	breakdown:requests_by_borough
Demandes pour 10 000 habitants par borough	6 modalités ou moins : colonnes	breakdown:requests_per_10k_by_borough
Délai médian de résolution (heures) par borough, demandes clôturées	6 modalités ou moins : colonnes	breakdown:median_resolution_hours_by_borough
Nombre de demandes par type de plainte	plus de 6 modalités : barres horizontales, top 15	breakdown:requests_by_complaint_type
% de demandes clôturées sous le délai cible, par type de plainte	6 modalités ou moins : colonnes	breakdown:sla_compliance_by_complaint_type
Nombre de demandes par jour de création	dimension temporelle : courbe avec anomalies	breakdown:daily_requests
Réponses aux questions métier	réponses aux questions : table	question:1, question:2

Panneau Dashboard Generator : chaque élément, la règle qui l'a choisi et ses requêtes d'origine.

Ce que je dis (environ 3 minutes avec le scénario)

> « Je suis le Dashboard Generator. Je reçois les KPI, les réponses et les insights, et je décide comment les montrer.
>
> Je n'utilise pas de LLM : je suis des règles simples et justifiables. Un KPI devient une carte ; une série par jour devient une courbe avec les anomalies en rouge ; une dimension avec peu de valeurs devient des barres verticales, avec beaucoup de valeurs des barres horizontales. Ici : 13 éléments.
>
> Le point important : chaque graphique chiffré garde la liste des requêtes qui l'ont produit. Il n'existe aucun graphique sans requête d'origine ; sinon, ma validation échoue. C'est ce qui permet le « Voir le SQL » sous chaque chiffre.
>
> Et je ne publie rien moi-même. J'écris une spécification avec une empreinte. La publication n'a lieu qu'après l'Auditeur. Ensuite, à chaque affichage, l'application recalcule l'empreinte : si quelqu'un modifie un chiffre dans le fichier après l'audit, le tableau de bord refuse de l'afficher. [Montrer « Spec vérifiée » sur le tableau de bord.]
>
> Mon scénario montre ce qui arrive si ma spécification est fausse. [Lancer *Écart persistant* sur Dashboard Generator.] L'Auditeur signale mon étape : la flèche me renvoie le run, je refais la spécification, il contrôle une seconde fois. L'écart est toujours là : le run est déclaré **non fiable** et rien n'est publié. Le tableau de bord garde le dernier run validé. [Montrer le Tableau de bord, puis l'Historique.] »



Scénario 5 : écart persistant

Renvoi vers Dashboard Generator, second audit toujours en écart : la ligne s'arrête sur « Bloqué », le run est **Non fiable**.

Quand ça tourne mal

Cas	Ce qui se passe
Spécification modifiée à la main après l'audit	empreinte différente : l'application refuse de l'afficher (testé automatiquement)
Run en échec ou non fiable	la publication n'est pas mise à jour : le tableau de bord reste sur le dernier run validé
Graphique sans requête d'origine	validation en échec : 3 essais, puis **Échec**
L'Auditeur signale un écart sur cette étape	c'est mon scénario 5 : un renvoi, puis **Non fiable** si l'écart reste

Questions possibles du jury

Pourquoi pas un LLM pour choisir les graphiques ?

Le choix doit être reproductible et explicable : la même donnée donne toujours le même graphique.

Qui décide de publier ?

Personne d'autre que le résultat de l'audit : l'orchestrateur publie seulement si l'Auditeur approuve.

Pourquoi pas Power BI ou Tableau ?

Ils ne sont pas pilotables par un agent ; ici la spécification est générée, vérifiée et affichée par notre application.

6 Auditeur / XAI

Présenté par _____

Je rejoue et je vérifie chaque chiffre avant publication, et j'explique en clair d'où il vient. C'est le code qui vérifie, pas un LLM.

Mon scénario **6. Écart corrigé** — page Scénarios, environ 2 min 30, Publié

Déclencheur	sortie Dashboard validée
Entrées	tout l'état du run et l'instantané
Sortie	verdict, contrôles, écarts adressés à l'étape fautive, explication
LLM	aucun
Code	`src/biflow/agents/auditor_xai.py`

Le scénario, étape par étape

1. **Données** : l'empreinte de l'instantané est recalculée, le nombre de lignes concorde.
2. **KPI** : chaque KPI affiché = résultat enregistré de sa requête.
3. **Rejeu** de toutes les requêtes : même résultat attendu.
4. **Recalcul indépendant** de 4 KPI en pandas, sans le SQL du catalogue.
5. **Confidentialité** : aucun résultat sur moins de 10 demandes.
6. **Insights** : chaque nombre cité est dans les preuves.
7. **Tableau de bord** : empreinte et valeurs affichées.
8. Verdict : aucun écart bloquant → Publié.

Les chiffres à retenir

53 / 53
contrôles réussis sur le run réel

15
requêtes rejouées

18, 20, 12, 3
contrôles sur KPI, Analyste, Dashboard et Qualité

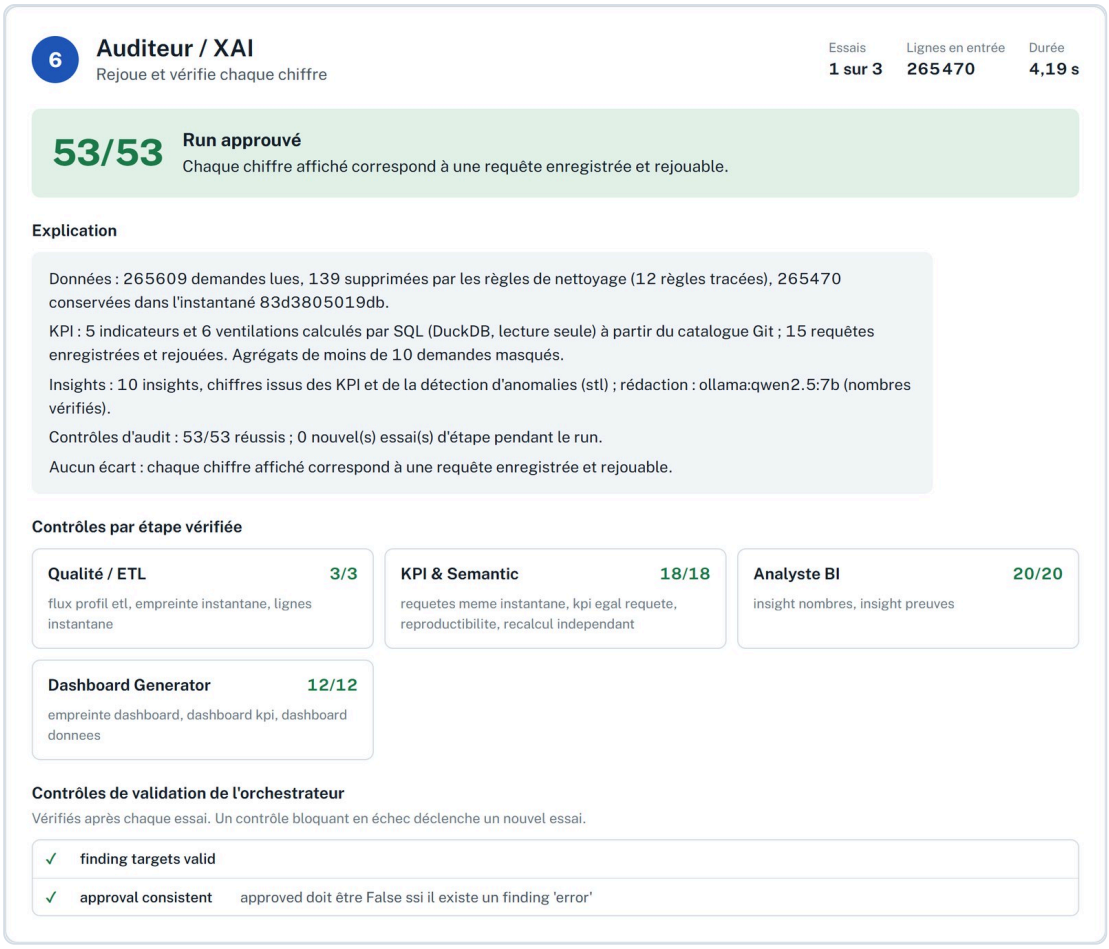
4
KPI recalculés indépendamment en pandas

4 / 4
erreurs volontaires bloquées à l'évaluation

1
seul renvoi autorisé, puis « non fiable »

Ce que je montre

1. Onglet **6 Auditeur / XAI** : 53/53, l'explication en clair, les contrôles par étape.
2. **Scénario 6** : *Écart corrigé par l'Auditeur*, étape **KPI & Semantic**, **Lancer** (environ 2 min 30). Pendant le run, commenter l'onglet Auditeur.
3. À la fin : la flèche de renvoi, « 2 passages », le second audit 53/53, « Publié ».
4. Rappeler le scénario 5 : même renvoi, mais l'écart restait, donc rien n'a été publié.



Panneau Auditeur / XAI : verdict, explication et contrôles par étape vérifiée.

Ce que je dis (environ 3 minutes, pendant le scénario)

> « Je suis l'Auditeur, le dernier agent, et le seul qui peut laisser publier. Je ne fais pas confiance aux autres agents : je refais leurs vérifications.
>
> Je recalcule l'empreinte des données, je rejoue les 15 requêtes enregistrées, je recalcule 4 KPI d'une autre façon, en pandas, et je vérifie que chaque nombre cité dans un insight existe dans les preuves. Ici : 53 contrôles sur 53, et une explication en français que n'importe qui peut lire.
>
> Que se passe-t-il si je trouve un écart ? C'est mon scénario, « Écart corrigé ». [Lancé sur KPI & Semantic au début de ma partie.] Je signale un KPI qui ne correspond pas : la flèche renvoie le run vers l'agent KPI. Il refait son travail, et les agents suivants aussi : on voit « 2 passages ». Puis je contrôle une seconde fois : 53 sur 53, et le run est publié.
>
> Le scénario 5 a montré l'autre cas : si l'écart reste, le run est non fiable et rien n'est publié. Nous n'autorisons qu'un seul renvoi, pour éviter une boucle sans fin : au-delà, un humain doit regarder. »



Scénario 6 : écart corrigé
Renvoi vers KPI & Semantic, les étapes suivantes refont leur travail (« 2 passages »), le second audit passe : Publié.

Les écarts que l'Auditeur sait renvoyer

Écart détecté	Renvoyé vers
Instantané modifié entre l'ETL et l'audit	Qualité / ETL
KPI affiché différent du résultat de sa requête	KPI & Semantic
Insight avec un nombre absent des preuves	Analyste BI
Spécification du tableau de bord modifiée	Dashboard Generator

Questions possibles du jury

Pourquoi pas un LLM pour auditer ?
Un audit doit être exact et reproductible ; c'est du code, testé, qui compare des valeurs.

L'écart est simulé : qu'est-ce que ça prouve ?
Que le circuit fonctionne : renvoi, réexécution, second audit, refus de publier. À l'évaluation, 4 erreurs réelles injectées sur 4 ont été bloquées.

Pourquoi un seul renvoi ?
Pour éviter une boucle infinie : si une correction automatique ne suffit pas, il faut un humain.

Questions transverses

N'importe quel membre peut recevoir ces questions : elles portent sur l'ensemble du projet.

Question	Réponse courte
Pourquoi des agents et pas un seul programme ?	Chaque rôle a une entrée, une sortie et des contrôles : on peut tester, relancer et auditer chaque étape séparément.
Pourquoi LangGraph ?	Il gère l'état partagé, les relances et la boucle de l'Auditeur de façon explicite ; CrewAI ou AutoGen laissent l'outil décider de l'ordre, difficile à justifier pour des chiffres officiels.
Combien d'agents utilisent un LLM ?	Deux sur six : KPI & Semantic (question → SQL) et Analyste BI (rédaction). Tout est contrôlé par le code.
Et sans LLM ?	Tout fonctionne avec la baseline à règles : 61 % des questions au lieu de 88 %, les mêmes KPI, le même audit.
RGPD ?	Texte libre retiré, coordonnées à environ 1 km, seuil de 10 demandes, question contrôlée avant envoi, jamais de ligne brute au LLM, modèle local possible (aucune donnée ne sort).
Pourquoi 2 % des données ?	Acquisition bornée (45 mois en environ 265 000 lignes) ; échantillon non probabiliste, d'où la réserve affichée sur le tableau de bord.
Comment savoir que les chiffres sont justes ?	L'Auditeur rejoue tout ; et le BI Analyst de l'équipe, code indépendant, trouve le même borough au délai médian le plus long (Staten Island, 15,736 h).
Combien de temps prend un run ?	Environ 10 s sans LLM, environ 1 min 30 avec le LLM local (surtout la rédaction des insights).
Comment est-ce testé ?	95 tests automatiques, dont les 4 scénarios de bout en bout ; ils tournent sur GitHub à chaque modification.
Quelles sont les limites ?	Échantillon non probabiliste, délais cibles provisoires, LLM mesuré avec le modèle local seulement, pannes simulées pour la démonstration.

Où trouver le reste

Document	Contenu
`README.md`	installation, application, 4 scénarios, 6 agents, API, sécurité, tests
`docs/SCENARIOS.md`	chaque agent en détail, avec tous les cas d'erreur
`docs/DEMO.md`	déroulé de la démonstration et commandes de secours
`livrables/`	présentation, rapport (docx, pdf), vidéo de 4 min